DATA-260 Homework 9 – Report

Uday Patel

GitHub repository: `https://github.com/UdayPate/data260-9871`

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16" (Ryzen 5 7640HS, 16GB RAM, RTX 4060 8GB, 512GB SSD).
Local model used: qwen3:8b via Ollama (thinking off; sampling seed 9871 and temperature 0 in Part 4) for the HW8 agent pipeline (Part 2) and the Part 4 Answering Agent; sentence-transformers/all-MiniLM-L6-v2 for the embeddings.
Cache: Redis Stack 7.4.0 (`redis/redis-stack-server`, RediSearch vector index, FLAT, cosine) in docker compose; MySQL `s9871_rel`; every Redis key starts with `s9871:`.
Repository: `https://github.com/UdayPate/data260-9871`
Tagged commit: PLACEHOLDER_HASH (to be filled in after the final commit and hw9 tag)
`https://github.com/UdayPate/data260-9871/tree/PLACEHOLDER_HASH`
Collaborator access: confirmed working for Sbnikitha and supriyaselvanganesan.

Part 1 – Redis View Counter

Reuse: the primary entity is the `fixtures` table of `s9871_rel` (HW4/HW5); it gets a new `views` column (`migrate_hw09.py`). The one new method is `FixtureCache.increment_fixture_views(fixture_id)` in `code/cache_hw9/fixture_cache.py`: it `INCR`s the key `s9871:views:fixture:<id>` on every access, and when the count is a multiple of 10 it copies the count to MySQL and logs `Synced views to database`. Redis runs as a new service in the existing `docker-compose.yml`.

```
    def increment_fixture_views(self, fixture_id: int) -> int:
        if isinstance(fixture_id, bool) or not isinstance(fixture_id, int) or fixture_id < 1:
            raise ValueError(f"fixture_id must be a positive integer, got {fixture_id!r}")
        count = self.redis_client.incr(self.views_key(fixture_id))  # atomic, so concurrent accesses never lose a
      view
        if count % SYNC_EVERY == 0:
            try:
                with self.engine.begin() as conn:
                    result = conn.execute(text("UPDATE fixtures SET views = :views WHERE id = :id"),
                                          {"views": count, "id": fixture_id})
                if result.rowcount:
                    log.info("Synced views to database")
```

```
                else:
                    log.warning("Sync skipped: no fixture with id %s in the database (count kept in Redis)",
    fixture_id)
            except SQLAlchemyError as exc:  # counting a view must not fail because MySQL is down; the next sync
    catches up
                log.error("Sync to database failed for fixture %s at count %s: %s", fixture_id, count, type(exc).
    __name__)
            return count
```

Output of counter_demo.py (my own PowerShell, 25 accesses of fixture 5)

```
PS C:\Users\patel\data260-9871>  cd C:\Users\patel\data260-9871\code\cache_hw9
PS C:\Users\patel\data260-9871\code\cache_hw9> ..\venv\Scripts\python.exe counter_demo.py
=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
ok Synced views to database
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
ok Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis s9871:views:fixture:5 = 25
MySQL fixtures.views (fixture 5) = 20  (last sync was at 20; views 21-25 are only in Redis until the next multiple of 10)
PS C:\Users\patel\data260-9871\code\cache_hw9>
```

Both syncs fire exactly at accesses 10 and 20. The last two lines are read straight from Redis and MySQL after the run: Redis holds all 25 views, MySQL holds the last synced value (20); views 21-25 reach MySQL at the next multiple of 10.

Checked directly in Redis and MySQL, and by tests

```
$ docker exec s9871-redis redis-cli GET s9871:views:fixture:5
25
$ docker exec s9871-redis redis-cli --scan --pattern 's9871:*'
s9871:views:fixture:5
$ SELECT id, fixture_name, views FROM fixtures WHERE id IN (5, 6)   (MySQL s9871_rel, separate connection)
(5, 'Wolves vs Lions (Basketball #3)', 20)
(6, 'Lions vs Strikers (Soccer #4)', 0)
rows with views > 0: 1
```

`test_counter.py` (13 checks, real Redis and MySQL; the MySQL value is read after every access) passes 13/13. As a mutation check, syncing at the wrong moment made it fail (10/13); the failing lines:

```
[FAIL] MySQL changes only at 10 and 20 (read after every access)  (after 9=0, 10=0, 11=11, 20=11, 25=22)
[FAIL] unknown fixture id: counter still returns 1..10
[FAIL] MySQL down at the 10th access: returns 10 and logs an error
10/13 checks passed
```

The tests also cover the failure paths: ids 0, -5, '1', None, True and 1.5 raise `ValueError` and create no key; an unknown fixture id keeps counting but logs a warning at the sync and creates no row; with MySQL down the 10th access still returns 10 and logs an error; with Redis down the error reaches the caller.

Part 2 – Semantic Response Cache

Reuse: on a miss the question goes through the unchanged HW8 Planner/Writer/Reviewer pipeline over Kafka (`pipeline_client.py`), with the fixture facts for any `FX-9871-nnnnn` code in the question prepended from MySQL (`facts.py`). `CachedQA.ask` (`qa_service.py`) embeds the query with all-MiniLM-L6-v2, finds the nearest cached query with a RediSearch KNN query (FLAT index, cosine; similarity = 1 - distance), and returns `is_cached`, `similarity` and `latency_ms` with every answer. An entry stores the query, its embedding, the final answer, a status and a data-version marker (a SHA-256 fingerprint of the fixture facts the answer used; `updated_at` is deliberately not used because it changes on every Part 1 view sync). Only answers the Reviewer approved are cached.

```
    def ask(self, query: str, threshold: float | None = None) -> dict:
        threshold = self.threshold if threshold is None else threshold
        started_at, t0 = datetime.now().isoformat(timespec="milliseconds"), now_ms()
        vector = embed(query)
        t1 = now_ms()
        best = self.cache.nearest(vector)
        t2 = now_ms()
        timings = {"embed_ms": t1 - t0, "search_ms": t2 - t1}
        similarity = best["similarity"] if best else None

        stale = False
        if best and similarity >= threshold and self.check_version and best["refs"]:
            t_v = now_ms()
            current = fingerprint(fixture_facts(best["refs"]))
            timings["version_check_ms"] = now_ms() - t_v
            if current != best["version"]:
                stale = True
                self.cache.delete(best["key"])
                log.warning("CACHE STALE similarity=%.4f stored_version=%s current_version=%s refs=%s - entry %r invalidated",
                            similarity, best["version"], current, best["refs"], best["query"])

        if best and similarity >= threshold and not stale:
            log.info("CACHE HIT  similarity=%.4f threshold=%.2f query=%r matched=%r", similarity, threshold, query, best["query"])
            result = {"answer": best["answer"], "is_cached": True, "status": best["status"], "matched_query": best["query"],
                      "data_version": best["version"]}
        else:
            log.info("CACHE MISS best_similarity=%s threshold=%.2f query=%r - running the HW8 agent pipeline",
                     "n/a (empty cache)" if similarity is None else f"{similarity:.4f}", threshold, query)
            facts = lookup_facts(query)
            t3 = now_ms()
            final = self.agent_fn(facts["block"] + "Question: " + query)
            t4 = now_ms()
            timings["facts_ms"], timings["agent_ms"] = t3 - t2, t4 - t3
            cached = final.get("status") == "approved"
            if cached:
                self.cache.store(query, vector, final["answer"], facts["version"], facts["refs"])
            timings["store_ms"] = now_ms() - t4
            if not cached:
                log.info("not cached: pipeline status=%r", final.get("status"))
            result = {"answer": final.get("answer", ""), "is_cached": False, "status": final.get("status"), "matched_query": None,
                      "data_version": facts["version"], "stored_in_cache": cached, "invalidated_stale_entry": stale}
        total = now_ms() - t0
        return {"query": query, **result, "similarity": similarity, "threshold": threshold, "latency_ms": round(total, 1),
                "timings_ms": {k: round(v, 1) for k, v in timings.items()}, "started_at": started_at,
                "finished_at": datetime.now().isoformat(timespec="milliseconds")}
```

```
    def nearest(self, vector: np.ndarray):
        """Best cached entry for the vector: {key, similarity, query, answer, version, refs, status}, or None when the cache is empty."""
        q = (Query("*=>[KNN 1 @embedding $vec AS dist]").sort_by("dist")
             .return_fields("query", "answer", "version", "refs", "status", "dist").dialect(2))
        docs = self.r.ft(INDEX).search(q, query_params={"vec": vector.tobytes()}).docs
        if not docs:
            return None
        d = docs[0]
        return {"key": d.id, "similarity": round(1.0 - float(d.dist), 4), "query": d.query, "answer": d.answer,
                "version": d.version, "refs": json.loads(d.refs), "status": d.status}

    def store(self, query: str, vector: np.ndarray, answer: str, version: str, refs: list[str], status: str = "approved") -> str:
        key = f"{KEY_PREFIX}{uuid.uuid4().hex[:12]}"
        self.r.hset(key, mapping={"query": query, "embedding": vector.tobytes(), "answer": answer, "status": status,
                                  "version": version, "refs": json.dumps(refs), "created_at": datetime.now().isoformat(timespec="seconds")})
        return key
```

Test: 13 queries (5 new, 4 exact duplicates, 4 paraphrases), run in my own PowerShell on the real pipeline (qwen3:8b through Kafka); one uncounted warm-up question loaded the model first. Each log line below is written before the answer is returned.

```
2026-11-06 15:20:12.369 CACHE MISS best_similarity=n/a (empty cache) threshold=0.85 query='Which team is the home team for fixture FX-9871-00005?' -
      running the HW8 agent pipeline
2026-11-06 15:20:17.558 CACHE HIT  similarity=1.0000 threshold=0.85 query='Which team is the home team for fixture FX-9871-00005?' matched='Which team
      is the home team for fixture FX-9871-00005?'
```

```
2026-11-06 15:20:17.570 CACHE MISS best_similarity=0.8051 threshold=0.85 query='Who is playing in fixture FX-9871-00006?' - running the HW8 agent
      pipeline
2026-11-06 15:20:22.887 CACHE HIT  similarity=0.8824 threshold=0.85 query='Tell me which team hosts fixture FX-9871-00005 at home.' matched='Which team
       is the home team for fixture FX-9871-00005?'
2026-11-06 15:20:22.897 CACHE MISS best_similarity=0.1825 threshold=0.85 query='What are the basic rules of volleyball scoring?' - running the HW8
      agent pipeline
2026-11-06 15:20:29.702 CACHE HIT  similarity=1.0000 threshold=0.85 query='Who is playing in fixture FX-9871-00006?' matched='Who is playing in fixture
       FX-9871-00006?'
2026-11-06 15:20:29.714 CACHE HIT  similarity=0.8872 threshold=0.85 query='Which two teams are playing in fixture FX-9871-00006?' matched='Which team
      is the home team for fixture FX-9871-00005?'
2026-11-06 15:20:29.727 CACHE MISS best_similarity=0.6700 threshold=0.85 query='How many player spots are still available for fixture FX-9871-00007?' -
       running the HW8 agent pipeline
2026-11-06 15:20:34.304 CACHE HIT  similarity=1.0000 threshold=0.85 query='What are the basic rules of volleyball scoring?' matched='What are the basic
       rules of volleyball scoring?'
2026-11-06 15:20:34.315 CACHE HIT  similarity=0.8714 threshold=0.85 query='How does scoring work in volleyball?' matched='What are the basic rules of
      volleyball scoring?'
2026-11-06 15:20:34.328 CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are still available for fixture FX-9871-00007?'
      matched='How many player spots are still available for fixture FX-9871-00007?'
2026-11-06 15:20:34.340 CACHE MISS best_similarity=0.7812 threshold=0.85 query='How many open player slots are left on fixture FX-9871-00007?' -
      running the HW8 agent pipeline
2026-11-06 15:20:38.901 CACHE HIT  similarity=0.8708 threshold=0.85 query='What is the home ground of the home team for fixture FX-9871-00008?' matched
      ='Which team is the home team for fixture FX-9871-00005?'
```

#	kind	query	best similarity	result	latency (ms)
1	new	Which team is the home team for fixture FX-9871-00005?	n/a	MISS	5192.0
2	duplicate	Which team is the home team for fixture FX-9871-00005?	1.0000	HIT	18.2
3	new	Who is playing in fixture FX-9871-00006?	0.8051	MISS	5314.4
4	paraphrase	Tell me which team hosts fixture FX-9871-00005 at home.	0.8824	HIT	14.8
5	new	What are the basic rules of volleyball scoring?	0.1825	MISS	6800.9
6	duplicate	Who is playing in fixture FX-9871-00006?	1.0000	HIT	13.8
7	paraphrase	Which two teams are playing in fixture FX-9871-00006?	0.8872	HIT (false hit)	11.7
8	new	How many player spots are still available for fixture FX-9871-00007?	0.6700	MISS	4577.2
9	duplicate	What are the basic rules of volleyball scoring?	1.0000	HIT	13.3
10	paraphrase	How does scoring work in volleyball?	0.8714	HIT	10.5
11	duplicate	How many player spots are still available for fixture FX-9871-00007?	1.0000	HIT	11.9
12	paraphrase	How many open player slots are left on fixture FX-9871-00007?	0.7812	MISS	4558.4
13	new	What is the home ground of the home team for fixture FX-9871-00008?	0.8708	HIT (false hit)	14.8

Metric	Value
Queries	13
Cache hits / misses	8 / 5
Hit rate	61.5%
Mean latency, cached	13.6 ms
Mean latency, not cached	5288.6 ms
Speed-up (not cached / cached)	389x
False hits	2 (queries #7, #13)
Exact duplicates served from cache	4 of 4
Paraphrases served from cache	3 of 4
New questions that hit	1 of 5

What this shows: a cache hit costs one embedding plus one vector search (about 13.6 ms), while a miss runs the whole Planner/Writer/Reviewer pipeline over Kafka with qwen3 (about 5.3 s), so a hit is roughly 389x faster. Every exact duplicate hit (similarity 1.0000). Paraphrases hit when similarity reached 0.85 and missed when it did not (#12, 0.7812).

The cache also made mistakes. Each hit above is judged against a label fixed before the run: a query that asks about a different fixture than the cached one is a false hit. Query #7 (fixture 00006, served the fixture 00005 answer, similarity 0.8872) and query #13 (fixture 00008, same wrong entry, 0.8708) are false hits. The embedding model sees "FX-9871-00005" and "FX-9871-00006" as almost the same text, so the fixture code barely moves the similarity, and it is as high as a true paraphrase (#4 at 0.8824, #10 at 0.8714). That is the threshold trade-off Part 3 examines; no threshold on this run separates the true paraphrases from the false hits cleanly, so nothing was relabelled or tuned here.

Part 3 – Threshold Economics and Stale Answers

3.1 Labeled test set (cache_cases.yaml)

Committed (commit `1c72c91`) before any threshold was run and not edited afterwards. Ten seed queries sit in the cache; each of the 40 cases is looked up against them. `validate_cases.py` checks the labels against the database without using any embedding (10/10 checks, and a mutation check caught a case whose answer was not actually different).

```
# DATA-260 HW9 Part 3.1 - labeled cache test set (SID4 9871, domain: community sports league fixtures)
# Written and committed BEFORE any threshold sweep. Labels are never edited after results; a labeling mistake
# would be documented separately (reports/hw09/LABEL_NOTES.md), not fixed here.
#
# How it is used: the 10 ‘seeds‘ are the queries already in the cache (each stored with its correct answer).
# Each of the 40 ‘cases‘ is then looked up against that cache:
#   expected: hit   -> the case should be served by the cache entry named in ‘matches‘ (and no other entry)
#   expected: miss  -> the case must NOT be served from the cache, because the correct answer differs / is unrelated
# A false hit = a case labeled ‘miss‘ that was served from the cache, or a ‘hit‘ case served by the wrong entry.
# For ‘different_answer‘ cases ‘differs_from‘ is the seed whose cached answer would be wrong; the correct answers
# differ in s9871_rel (checked by validate_cases.py, which uses no embeddings and no similarity).
# attribute: home_team | teams | home_ground | fixture_name | spots (what the question asks about the fixture)

seeds:
  - {id: S01, fixture: FX-9871-00005, attribute: home_team,    query: "Which team is the home team for fixture FX-9871-00005?"}
  - {id: S02, fixture: FX-9871-00006, attribute: teams,        query: "Who is playing in fixture FX-9871-00006?"}
  - {id: S03, fixture: FX-9871-00007, attribute: home_ground,  query: "What is the home ground for fixture FX-9871-00007?"}
  - {id: S04, fixture: FX-9871-00008, attribute: fixture_name, query: "What is the name of fixture FX-9871-00008?"}
  - {id: S05, fixture: FX-9871-00009, attribute: home_team,    query: "Which team is the home team for fixture FX-9871-00009?"}
  - {id: S06, fixture: FX-9871-00010, attribute: home_ground,  query: "What is the home ground for fixture FX-9871-00010?"}
  - {id: S07, fixture: FX-9871-00011, attribute: teams,        query: "Who is playing in fixture FX-9871-00011?"}
  - {id: S08, fixture: FX-9871-00012, attribute: fixture_name, query: "What is the name of fixture FX-9871-00012?"}
  - {id: S09, fixture: FX-9871-00013, attribute: spots,        query: "How many player spots are available for fixture FX-9871-00013?"}
  - {id: S10, fixture: FX-9871-00014, attribute: home_team,    query: "Which team is the home team for fixture FX-9871-00014?"}

cases:
  # ---- 10 exact duplicates (should hit their own seed) ----
  - {id: C01, category: exact_duplicate, expected: hit, matches: S01, query: "Which team is the home team for fixture FX-9871-00005?"}
  - {id: C02, category: exact_duplicate, expected: hit, matches: S02, query: "Who is playing in fixture FX-9871-00006?"}
  - {id: C03, category: exact_duplicate, expected: hit, matches: S03, query: "What is the home ground for fixture FX-9871-00007?"}
  - {id: C04, category: exact_duplicate, expected: hit, matches: S04, query: "What is the name of fixture FX-9871-00008?"}
  - {id: C05, category: exact_duplicate, expected: hit, matches: S05, query: "Which team is the home team for fixture FX-9871-00009?"}
  - {id: C06, category: exact_duplicate, expected: hit, matches: S06, query: "What is the home ground for fixture FX-9871-00010?"}
  - {id: C07, category: exact_duplicate, expected: hit, matches: S07, query: "Who is playing in fixture FX-9871-00011?"}
  - {id: C08, category: exact_duplicate, expected: hit, matches: S08, query: "What is the name of fixture FX-9871-00012?"}
  - {id: C09, category: exact_duplicate, expected: hit, matches: S09, query: "How many player spots are available for fixture FX-9871-00013?"}
  - {id: C10, category: exact_duplicate, expected: hit, matches: S10, query: "Which team is the home team for fixture FX-9871-00014?"}

  # ---- 10 paraphrases that should hit (same fixture, same question, different words) ----
  - {id: C11, category: paraphrase, expected: hit, matches: S01, query: "Tell me which team hosts fixture FX-9871-00005 at home."}
  - {id: C12, category: paraphrase, expected: hit, matches: S02, query: "Which two teams are facing each other in fixture FX-9871-00006?"}
  - {id: C13, category: paraphrase, expected: hit, matches: S03, query: "Where is the home venue of fixture FX-9871-00007?"}
  - {id: C14, category: paraphrase, expected: hit, matches: S04, query: "What is fixture FX-9871-00008 called?"}
  - {id: C15, category: paraphrase, expected: hit, matches: S05, query: "Name the home side for fixture FX-9871-00009."}
  - {id: C16, category: paraphrase, expected: hit, matches: S06, query: "At which ground does fixture FX-9871-00010 take place at home?"}
  - {id: C17, category: paraphrase, expected: hit, matches: S07, query: "Which teams are playing against each other in fixture FX-9871-00011?"}
  - {id: C18, category: paraphrase, expected: hit, matches: S08, query: "What title does fixture FX-9871-00012 have?"}
  - {id: C19, category: paraphrase, expected: hit, matches: S09, query: "How many open spots remain on fixture FX-9871-00013?"}
  - {id: C20, category: paraphrase, expected: hit, matches: S10, query: "Who is the home team of fixture FX-9871-00014?"}

  # ---- 10 superficially similar, different correct answer (should miss) ----
  # Same wording as a seed but a different fixture, or the same fixture but a different attribute.
  - {id: C21, category: different_answer, expected: miss, differs_from: S01, fixture: FX-9871-00015, attribute: home_team,    query: "Which team is the
      home team for fixture FX-9871-00015?"}
  - {id: C22, category: different_answer, expected: miss, differs_from: S02, fixture: FX-9871-00016, attribute: teams,        query: "Who is playing in
      fixture FX-9871-00016?"}
  - {id: C23, category: different_answer, expected: miss, differs_from: S03, fixture: FX-9871-00008, attribute: home_ground,  query: "What is the home
     ground for fixture FX-9871-00008?"}
  - {id: C24, category: different_answer, expected: miss, differs_from: S04, fixture: FX-9871-00010, attribute: fixture_name, query: "What is the name
     of fixture FX-9871-00010?"}
  - {id: C25, category: different_answer, expected: miss, differs_from: S05, fixture: FX-9871-00011, attribute: home_team,    query: "Which team is the
      home team for fixture FX-9871-00011?"}
  - {id: C26, category: different_answer, expected: miss, differs_from: S06, fixture: FX-9871-00012, attribute: home_ground,  query: "What is the home
     ground for fixture FX-9871-00012?"}
```

```
  - {id: C27, category: different_answer, expected: miss, differs_from: S07, fixture: FX-9871-00005, attribute: teams,        query: "Who is playing in
      fixture FX-9871-00005?"}
  - {id: C28, category: different_answer, expected: miss, differs_from: S08, fixture: FX-9871-00014, attribute: fixture_name, query: "What is the name
      of fixture FX-9871-00014?"}
  - {id: C29, category: different_answer, expected: miss, differs_from: S09, fixture: FX-9871-00013, attribute: home_ground,  query: "What is the home
      ground for fixture FX-9871-00013?"}
  - {id: C30, category: different_answer, expected: miss, differs_from: S10, fixture: FX-9871-00018, attribute: home_team,    query: "Which team is the
      home team for fixture FX-9871-00018?"}

  # ---- 10 unrelated questions (should miss) ----
  - {id: C31, category: unrelated, expected: miss, query: "What are the basic rules of volleyball scoring?"}
  - {id: C32, category: unrelated, expected: miss, query: "How do I register my own team for next season?"}
  - {id: C33, category: unrelated, expected: miss, query: "What should I eat before an early morning match?"}
  - {id: C34, category: unrelated, expected: miss, query: "How long is a standard soccer game?"}
  - {id: C35, category: unrelated, expected: miss, query: "What is the weather forecast for Sunnyvale this weekend?"}
  - {id: C36, category: unrelated, expected: miss, query: "Can you recommend a good pair of running shoes?"}
  - {id: C37, category: unrelated, expected: miss, query: "How do I stretch my hamstrings to avoid injury?"}
  - {id: C38, category: unrelated, expected: miss, query: "Explain what a referee does in cricket."}
  - {id: C39, category: unrelated, expected: miss, query: "What is the capital of France?"}
  - {id: C40, category: unrelated, expected: miss, query: "Write a short poem about autumn."}
```

3.2 Threshold sweep

The 40 labels in `code/cache_hw9/cache_cases.yaml` were committed (commit 1c72c91) before any threshold was run; they were not edited afterwards. The cache held the 10 seed queries. Run at 2026-11-06T15:49:43 (local time). A *false hit* is a case labeled "miss" that was served from the cache, or a "hit" case served by the wrong entry. False-hit rate = false hits / hits (the share of cached answers that are wrong). Latency = measured lookup (embedding + Redis search) for a hit; for a miss, lookup plus the measured mean pipeline time of the 5 non-cached queries of the real Part 2 run (5275.1 ms, `reports/hw09/raw/part2_results.json`) - the pipeline itself was not re-run 200 times.

Threshold	Hit rate	False-hit rate (false hits / hits)	Mean latency (ms)	Total cost (USD)
0.75	75.0% (30/40)	50.0% (15/30)	1326.9	37.60
0.80	70.0% (28/40)	50.0% (14/28)	1592.4	35.12
0.85	65.0% (26/40)	46.2% (12/26)	1854.2	30.14
0.90	60.0% (24/40)	45.8% (11/24)	2117.8	27.66
0.95	55.0% (22/40)	45.5% (10/22)	2382.4	25.18

Where the hits come from (same run):

Threshold	exact duplicates hit	paraphrases hit	different-answer hit (all false)	unrelated hit (all false)	true hits wrongly missed	misses
0.75	10/10	10/10	10/10	0/10	0	10
0.80	10/10	8/10	10/10	0/10	2	12
0.85	10/10	6/10	10/10	0/10	4	14
0.90	10/10	4/10	10/10	0/10	6	16
0.95	10/10	2/10	10/10	0/10	8	18

What this shows: the different-answer questions are *more* similar to the cached entry than the true paraphrases are, so raising the threshold removes true hits (0 lost at 0.75, 8 at 0.95) while the false hits barely move (15 at 0.75, 10 at 0.95). At 0.95, 10 of the 10 different-answer questions are still served a wrong cached answer. No threshold in this range separates the two groups. The false hits beyond the different-answer ones (at 0.75: 5) are paraphrases that were served by the wrong cached entry (a lower threshold lets a nearer-but-wrong seed win).

```
PS C:\Users\patel\data260-9871\code\cache_hw9>  ..\venv\Scripts\python.exe sweep_part3.py
threshold 0.75: hits=30/40 false_hits=15 missed_true_hits=0 mean_latency=1326.9 ms
threshold 0.80: hits=28/40 false_hits=14 missed_true_hits=2 mean_latency=1592.4 ms
threshold 0.85: hits=26/40 false_hits=12 missed_true_hits=4 mean_latency=1854.2 ms
threshold 0.90: hits=24/40 false_hits=11 missed_true_hits=6 mean_latency=2117.8 ms
threshold 0.95: hits=22/40 false_hits=10 missed_true_hits=8 mean_latency=2382.4 ms
wrote C:\Users\patel\data260-9871\reports\hw09\raw\part3_sweep.json
```

3.3 One false hit in full

- Cached query: `What is the home ground for fixture FX-9871-00007?`
- Incoming query: `What is the home ground for fixture FX-9871-00008?`
- Similarity: **0.999** (threshold 0.85, so it was served from the cache)
- Cached answer returned: The home ground is Cardoza Park, Milpitas.
- Correct answer (read from s9871_rel): The home ground is Backesto Park, San Jose.
- Why the model confuses them: the two sentences have 17 word-pieces each and differ only in 1 of them: `##7` vs `##8`. all-MiniLM-L6-v2 encodes the meaning of the sentence (a question about the home ground of a fixture); the fixture code is just a few digit pieces the model has no reason to treat as the part that decides the answer. For scale, a genuine paraphrase pair (C14: "What is fixture FX-9871-00008 called?" vs the cached "What is the name of fixture FX-9871-00008?") scores only 0.9626, lower than this false hit.

3.4 Cost model and threshold choice

Total cost over the 40 cases = (cases not served from cache) x C_MISS + (false hits) x C_FALSE, with C_MISS = 0.01 USD and C_FALSE = 2.5 USD.

- C_MISS = 0.01 USD: a miss runs the HW8 pipeline (three qwen3 calls plus Kafka and a database lookup, about 5.3 s of GPU time here); priced at an API-equivalent ~0.003 USD per model call, three calls come to about 0.01 USD. A hit costs about nothing, so this is what a hit saves.
- C_FALSE = 2.5 USD: a false hit gives the user a confident wrong fixture fact; the cost is one support contact, about 5 minutes of league-staff time at 30 USD/hour. It ignores the cost of a player turning up at the wrong ground, so it is on the low side.

Both are assumptions (`code/cache_hw9/cost_model.py`), so the cheapest threshold is also computed for other values of C_FALSE:

Threshold to ship: 0.95 (total 25.18 USD, the lowest in the sweep). A false hit is 250x as costly as a miss, so the cheapest choice is the highest threshold: it loses 8 true hits but avoids the most wrong answers. It is not a good result in absolute terms: 10 false hits remain at 0.95, because the embedding cannot tell one fixture code from another. Raising the threshold is the wrong tool for that failure; it needs a structural guard (for example, only serve a hit if the fixture codes in both queries are identical), which was not implemented because the assignment asks only to choose a threshold.

C_FALSE (USD)	cheapest threshold	its total (USD)
0.01	0.75	0.25
0.05	0.95	0.68
0.25	0.95	2.68
2.5	0.95	25.18
25	0.95	250.18

```
def total_cost(row: dict, c_miss: float = C_MISS, c_false: float = C_FALSE) -> float:
    return round(row["misses"] * c_miss + row["false_hits"] * c_false, 2)
```

3.5 Stale answers and invalidation

The check added to `CachedQA.ask` (off by default, so Part 2 behaves as measured; `check_version=True` in this demonstration):

```
        stale = False
        if best and similarity >= threshold and self.check_version and best["refs"]:
            t_v = now_ms()
            current = fingerprint(fixture_facts(best["refs"]))
            timings["version_check_ms"] = now_ms() - t_v
            if current != best["version"]:
                stale = True
                self.cache.delete(best["key"])
                log.warning("CACHE STALE similarity=%.4f stored_version=%s current_version=%s refs=%s - entry %r
    invalidated",
                            similarity, best["version"], current, best["refs"], best["query"])
```

Cached query: `How many player spots are available for fixture FX-9871-00013?` - depends on `fixtures.spots_available` of the single row FX-9871-00013. The row was changed from 22 to 17 and restored to 22 at the end.

Step	Result
1. first ask is a miss and the answer is cached	PASS - 22 player spots are available.
2. repeat is a hit with the same answer	PASS - similarity=1.0
3a. control: a view-counter sync (updated_at changes) does NOT invalidate	PASS - updated_at 2026-11-06 15:43:10 -> 2026-11-06 15:52:00, still a hit
3b. control: editing a different fixture does NOT invalidate	PASS -
4. database now says the new value (read directly)	PASS - 17 player spots are available.
5. STALE HIT without a version check: cache still returns the old answer	PASS - cache says '22 player spots are available.'; database says '17 player spots are available.'
6. with the version check: stale entry detected, pipeline re-run, fresh answer	PASS - new answer '17 player spots are available.', version 3878640e93bca289 -> 79ba45dd2a83ffc2
7. repeat is a hit with the fresh answer; exactly one entry (stale one deleted)	PASS - entries=1
8. cost of the check measured (30 hits each)	PASS - mean hit latency 8.32 ms without, 9.02 ms with

Strategy: version check on hit. Each entry stores a data-version marker (a SHA-256 fingerprint of the fixture fields the answer was built from). On a hit, the handler recomputes the fingerprint of the entry's rows from MySQL; if it differs, the entry is deleted and the question goes through the pipeline again (step 6: version 3878640e93bca289 -> 79ba45dd2a83ffc2). The marker is a content fingerprint, not `updated_at`, because `updated_at` also changes on every Part 1 view-counter sync (control 3a: it changed and the entry was correctly kept).

Trade-off: it never serves a stale answer for the fields it fingerprints, and it does not invalidate on unrelated edits, but every hit now pays a database read and the cache depends on MySQL being reachable: mean hit latency was 8.32 ms without the check and 9.02 ms with it (30 hits each), and a hit still cannot be served from Redis alone. A TTL would be cheaper per hit but would leave a window in which stale answers are served.

Terminal output of `stale_demo.py` (my own PowerShell; the 30+30 timing hits are only in the log file `raw/part3_stale.log`):

```
CACHE MISS best_similarity=n/a (empty cache) threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' - running the HW8
      agent pipeline
[PASS] 1. first ask is a miss and the answer is cached  22 player spots are available.
CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' matched='How many player spots are
      available for fixture FX-9871-00013?'
[PASS] 2. repeat is a hit with the same answer  similarity=1.0
CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' matched='How many player spots are
      available for fixture FX-9871-00013?'
[PASS] 3a. control: a view-counter sync (updated_at changes) does NOT invalidate  updated_at 2026-11-06 15:43:10 -> 2026-11-06 15:52:00, still a hit
CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' matched='How many player spots are
      available for fixture FX-9871-00013?'
[PASS] 3b. control: editing a different fixture does NOT invalidate
[PASS] 4. database now says the new value (read directly)  17 player spots are available.
CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' matched='How many player spots are
      available for fixture FX-9871-00013?'
[PASS] 5. STALE HIT without a version check: cache still returns the old answer  cache says '22 player spots are available.'; database says '17 player
      spots are available.'
CACHE STALE similarity=1.0000 stored_version=3878640e93bca289 current_version=79ba45dd2a83ffc2 refs=['FX-9871-00013'] - entry 'How many player spots
      are available for fixture FX-9871-00013?' invalidated
```

```
CACHE MISS best_similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' - running the HW8 agent
      pipeline
[PASS] 6. with the version check: stale entry detected, pipeline re-run, fresh answer  new answer '17 player spots are available.', version 3878640
      e93bca289 -> 79ba45dd2a83ffc2
CACHE HIT  similarity=1.0000 threshold=0.85 query='How many player spots are available for fixture FX-9871-00013?' matched='How many player spots are
      available for fixture FX-9871-00013?'
[PASS] 7. repeat is a hit with the fresh answer; exactly one entry (stale one deleted)  entries=1
[PASS] 8. cost of the check measured (30 hits each)  mean hit latency 8.32 ms without, 9.02 ms with
restored: FX-9871-00013 spots_available=22, views=0; cache cleared

9/9 checks passed
PS C:\Users\patel\data260-9871\code\cache_hw9>
```

Mutation check: with the staleness comparison disabled the demo failed steps 6 and 7 (7/9 passed, `raw/part3_stale_mutation_output.txt`).

Part 4 – Self-Improving Multi-Agent System

Model: qwen3:8b (Ollama, thinking off, temperature 0, seed 9871). Run at 2026-11-06T19:29:07 (local time). Input files: 18 test, 14 validation and 14 training questions (`code/agents_hw9/data/` , questions only; the reference dataset holds all fixtures of `s9871_rel`). The Evaluator is rule-based and reads only the reference dataset; the Answering Agent reads the same facts from MySQL and never sees the reference file.

Reference dataset and input files

`reference_dataset.json` holds every fixture of `s9871_rel` (5000 rows, one per line; first four shown). The Evaluator reads only this file. The three input files hold only an id and the question (no expected answers); no fixture appears in more than one of them. They were committed (commit `80fd59a`) before any result existed, and `test_data.json` was used only for the final test.


```
{
  "description": "Ground truth for the Part 4 Evaluator Agent: every league fixture exported from s9871_rel (fixtures JOIN teams). A fixture code that
      is not listed here does not exist in the league.",
  "source": "MySQL s9871_rel",
  "seed": 9871,
  "fixture_count": 5000,
  "fixtures": {
    "FX-9871-00004": {"fixture_name": "Strikers vs Hawks (Basketball #2)", "teams_players": "San Jose Strikers vs Santa Clara Hawks", "home_team": "
      Santa Clara Strikers", "home_ground": "Central Park Softball Field, Santa Clara", "spots_available": 22},
    "FX-9871-00005": {"fixture_name": "Wolves vs Lions (Basketball #3)", "teams_players": "San Jose Wolves vs Sunnyvale Lions", "home_team": "Sunnyvale
       Falcons", "home_ground": "Fair Oaks Park, Sunnyvale", "spots_available": 22},
    "FX-9871-00006": {"fixture_name": "Lions vs Strikers (Soccer #4)", "teams_players": "Milpitas Lions vs Santa Clara Strikers", "home_team": "Oakland
       Titans", "home_ground": "Mosswood Park, Oakland", "spots_available": 22},
    "FX-9871-00007": {"fixture_name": "Bears vs Sharks (Soccer #5)", "teams_players": "Sunnyvale Bears vs Oakland Sharks", "home_team": "Milpitas
      Eagles", "home_ground": "Cardoza Park, Milpitas", "spots_available": 22},
    ...
```


training_data.json (14)


```
[
  {"id": "tra_01", "question": "For fixture FX-9871-04418, which team is the home team and what is its home ground?"},
  {"id": "tra_02", "question": "Who is the referee for fixture FX-9871-01546?"},
  {"id": "tra_03", "question": "What is the name of fixture FX-9871-01574 and where is it played?"},
  {"id": "tra_04", "question": "How many player spots are available for fixture FX-9871-02996?"},
  {"id": "tra_05", "question": "Who is playing in fixture FX-9871-01988 and how many spots are left?"},
  {"id": "tra_06", "question": "What is the home ground of fixture FX-9871-01147 and of fixture FX-9871-03139?"},
  {"id": "tra_07", "question": "What is the capital of France?"},
  {"id": "tra_08", "question": "What time does fixture FX-9871-04498 kick off?"},
  {"id": "tra_09", "question": "Which team is the home team for fixture FX-9871-09101?"},
  {"id": "tra_10", "question": "At which venue is fixture FX-9871-00173 played?"},
  {"id": "tra_11", "question": "Who hosts fixture FX-9871-03205?"},
  {"id": "tra_12", "question": "Write a short poem about autumn."},
  {"id": "tra_13", "question": "What is the name of fixture FX-9871-04728?"},
  {"id": "tra_14", "question": "Who is playing in fixture FX-9871-01579?"}
]
```


validation_data.json (14)


```
[
  {"id": "val_01", "question": "What is the home ground of fixture FX-9871-04194 and of fixture FX-9871-00980?"},
  {"id": "val_02", "question": "Who is playing in fixture FX-9871-00910 and how many spots are left?"},
  {"id": "val_03", "question": "Who hosts fixture FX-9871-00345?"},
  {"id": "val_04", "question": "What is 15 times 12?"},
  {"id": "val_05", "question": "Which team is the home team for fixture FX-9871-09102?"},
```

```
  {"id": "val_06", "question": "For fixture FX-9871-01997, which team is the home team and what is its home ground?"},
  {"id": "val_07", "question": "What is the name of fixture FX-9871-00165?"},
  {"id": "val_08", "question": "What is the name of fixture FX-9871-03367 and where is it played?"},
  {"id": "val_09", "question": "What time does fixture FX-9871-03969 kick off?"},
  {"id": "val_10", "question": "What is the ticket price for fixture FX-9871-04515?"},
  {"id": "val_11", "question": "How many open slots are left on fixture FX-9871-02034?"},
  {"id": "val_12", "question": "Recommend a good pasta recipe."},
  {"id": "val_13", "question": "Who is playing in fixture FX-9871-04381?"},
  {"id": "val_14", "question": "What is the home ground for fixture FX-9871-01488?"}
]
```

test_data.json (18)

```
[
  {"id": "tes_01", "question": "Which team is the home team for fixture FX-9871-02934?"},
  {"id": "tes_02", "question": "Who is playing in fixture FX-9871-02271?"},
  {"id": "tes_03", "question": "For fixture FX-9871-00039, which team is the home team and what is its home ground?"},
  {"id": "tes_04", "question": "Translate 'good morning' into Spanish."},
  {"id": "tes_05", "question": "Explain how photosynthesis works."},
  {"id": "tes_06", "question": "What is the home ground of fixture FX-9871-00823 and of fixture FX-9871-04902?"},
  {"id": "tes_07", "question": "What is the name of fixture FX-9871-00914?"},
  {"id": "tes_08", "question": "What is the home team of fixture FX-9871-04960 and of fixture FX-9871-04971?"},
  {"id": "tes_09", "question": "Tell me the home team and the number of open slots for fixture FX-9871-02868."},
  {"id": "tes_10", "question": "What is the home ground for fixture FX-9871-00394?"},
  {"id": "tes_11", "question": "What is the ticket price for fixture FX-9871-02324?"},
  {"id": "tes_12", "question": "Who is playing in fixture FX-9871-01537 and how many spots are left?"},
  {"id": "tes_13", "question": "What is the name of fixture FX-9871-04813 and where is it played?"},
  {"id": "tes_14", "question": "Which team is the home team for fixture FX-9871-04308?"},
  {"id": "tes_15", "question": "What is the tallest mountain in the world?"},
  {"id": "tes_16", "question": "Which team is the home team for fixture FX-9871-09103?"},
  {"id": "tes_17", "question": "How many player spots are available for fixture FX-9871-02655?"},
  {"id": "tes_18", "question": "What was the final score of fixture FX-9871-02498?"}
]
```

The two agents

Reuse: `facts.py` from Part 2 gives the Answering Agent the same league facts from MySQL; `src/model_client.py` is the HW6 adapter. Agents exchange plain JSON messages through the controller `run_question` (below).

```
    def answer(self, msg: dict, forced_answer: str | None = None) -> dict:
        """msg = {"question", "attempt", "feedback", "previous_answer"}. forced_answer is the controlled-test hook: it replaces the model's
        first attempt with a given (deliberately incomplete or wrong) answer so the repair workflow can be shown reliably."""
        import time
        t0 = time.perf_counter()
        facts = lookup_facts(msg["question"])
        used = []
        system = SYSTEM
        if self.lessons is not None:
            used = self.lessons.relevant(question_tags(msg["question"], facts))
            if used:
                system += "\n\nLessons learned from earlier mistakes - follow them:\n" + "\n".join(f"- {ls['correction']}" for ls in used)
                for ls in used:
                    ls["use_count"] += 1
        user = facts["block"] + "Question: " + msg["question"]
        if msg.get("feedback"):
            user += (f"\n\nYour previous answer was:\n{msg['previous_answer']}\n\nThe evaluator rejected it with this feedback:\n{msg['feedback']}\n"
                     "Write a corrected answer (one short sentence).")
        if forced_answer is not None and msg["attempt"] == 1:
            text = forced_answer
        else:
            with contextlib.redirect_stdout(io.StringIO()):  # ModelClient prints token counts on every turn
                text = self.model.complete([{"role": "system", "content": system}, {"role": "user", "content": user}])
        text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL).strip()
        return {"answer": text, "attempt": msg["attempt"], "lessons_used": [ls["lesson_id"] for ls in used],
                "forced": forced_answer is not None and msg["attempt"] == 1, "latency_ms": round((time.perf_counter() - t0) * 1000, 1)}
```

```
    def evaluate(self, msg: dict) -> dict:
        question, answer = msg["question"], msg["answer"]
        a = _norm(answer)
        codes = fixture_codes(question)
        unknown = [c for c in codes if c not in self.ref]
        refuses = bool(REFUSAL.search(answer))
        # --- questions the reference cannot answer: unrelated, unknown fixture, or something the database does not record
        if not codes or unknown or asks_unrecorded(question):
            why = ("is not about league fixtures" if not codes else f"names fixture(s) {', '.join(unknown)} that are not in the reference data"
                   if unknown else "asks for something the league data does not record")
            leaked = [v for v in self.team_and_ground_values if v in a]
            if refuses and not leaked:
                return self._verdict("PASS", f"The question {why}, and the answer correctly says the information is not available.")
            return self._verdict("REVISE", f"The question {why}, but the answer gives an unsupported answer.",
                                 "The reference data has no answer to this question. Say that you do not have that information; do not answer from "
                                 "general knowledge and do not state any team or ground.", "UNSUPPORTED_ANSWER")
        # --- known fixtures: every requested fact must be present and correct
        attrs = attributes_asked(question)
        missing, wrong = [], []
```

```
        stripped = re.sub(r"FX-9871-\d{5}|#\d+", "", answer)
        numbers = set(re.findall(r"(?<![\w-])\d+(?![\w-])", stripped))
        for c in codes:
            for attr in attrs:
                for v in values_for(self.ref[c], attr):
                    if (v not in numbers) if attr == "spots" else (_norm(v) not in a):
                        missing.append((c, attr, v))
        for attr in attrs:
            if attr == "spots":
                allowed = {str(self.ref[c]["spots_available"]) for c in codes}
                wrong += [(",".join(codes), attr, n) for n in sorted(numbers - allowed)]
            else:
                wrong += [(",".join(codes), attr, f) for f in sorted(self._foreign_values(attr, codes)) if f in a]
        if wrong:
            detail = "; ".join(f"{attr.replace('_', ' ')}: '{w}' is not the value for the fixture asked about" for _, attr, w in wrong)
            fb = "Wrong value(s): " + detail + ". Reference: " + self._reference_text(codes, attrs)
            return self._verdict("REVISE", "The answer contains a value that does not match the reference data.", fb, "WRONG_FACT", wrong)
        if missing:
            miss_txt = "; ".join(f"{c} {attr.replace('_', ' ')} ('{v}')" for c, attr, v in missing)
            return self._verdict("REVISE", "The answer leaves out requested information: " + ", ".join(sorted({m[1].replace('_', ' ') for m in missing
      })) + ".",
                                 "Your answer is missing: " + miss_txt + ". Answer every part of the question using the facts provided.", "OMITTED_FACT
      ", missing)
        return self._verdict("PASS", "Every requested fact matches the reference data.")
```

```
def run_question(question: str, answering: AnsweringAgent, evaluator: EvaluatorAgent, use_feedback: bool, forced_first_answer: str | None = None) ->
      dict:
    t0 = time.perf_counter()
    attempts, feedback, previous = [], None, None
    for n in range(1, MAX_ATTEMPTS + 1):
        ans = answering.answer({"question": question, "attempt": n, "feedback": feedback, "previous_answer": previous}, forced_answer=
      forced_first_answer)
        ev = evaluator.evaluate({"question": question, "answer": ans["answer"]})
        attempts.append({"attempt": n, "answer": ans["answer"], "forced_for_test": ans["forced"], "lessons_used": ans["lessons_used"],
                         "answer_latency_ms": ans["latency_ms"], "evaluation": ev})
        if ev["result"] == "PASS" or not use_feedback or n == MAX_ATTEMPTS:
            break
        feedback, previous = ev["feedback"], ans["answer"]
    final = attempts[-1]
    return {"question": question, "attempts": attempts, "n_attempts": len(attempts), "final_answer": final["answer"],
            "final_result": final["evaluation"]["result"], "failure_type": final["evaluation"]["failure_type"],
            "latency_ms": round((time.perf_counter() - t0) * 1000, 1)}
```

4.1 Evaluator feedback and revision

- Question: For fixture FX-9871-00005, which team is the home team and what is its home ground?
- Attempt 1 (forced for the test): The home team is Sunnyvale Falcons.
- Evaluator: REVISE - The answer leaves out requested information: home ground.
- Feedback: Your answer is missing: FX-9871-00005 home ground ('Fair Oaks Park'). Answer every part of the question using the facts provided.
- Attempt 2 (revised by the Answering Agent): The home team for fixture FX-9871-00005 is Sunnyvale Falcons, and its home ground is Fair Oaks Park.
- Evaluator: PASS - Every requested fact matches the reference data.
- Attempts: 2 (maximum 2). Final answer / result: The home team for fixture FX-9871-00005 is Sunnyvale Falcons, and its home ground is Fair Oaks Park. / PASS


```
{
  "n_attempts": 2,
  "final_answer": "The home team for fixture FX-9871-00005 is Sunnyvale Falcons, and its home ground is Fair Oaks
      Park.",
  "final_result": "PASS"
}
```


4.2 Lessons saved and reused

Training (12/14 correct on the first attempt) failed on tra_07, tra_12 - the same pattern (UNSUPPORTED_ANSWER) twice, so the Evaluator proposed one lesson from it. A second candidate was

added on purpose as a controlled test of the validation gate.

lessons.json (kept store, after the demonstration):


```
[
  {
    "lesson_id": "lesson_001",
    "failure_pattern": "The answer states something the league facts do not support (unknown fixture, data the league does not record, or a question
      that is not about the league).",
    "correction": "If the provided facts do not contain what is asked, reply that you do not have that information; do not answer from general
      knowledge and do not guess.",
    "created_at": "2026-11-06",
    "use_count": 1,
    "failure_type": "UNSUPPORTED_ANSWER",
    "applies_when": [
      "no_data"
    ],
    "source": "training failures: tra_07, tra_12"
  }
]
```


lessons_candidates.json (all proposed lessons, used by the Lesson Memory variant):


```
[
  {
    "lesson_id": "lesson_001",
    "failure_pattern": "The answer states something the league facts do not support (unknown fixture, data the league does not record, or a question
      that is not about the league).",
    "correction": "If the provided facts do not contain what is asked, reply that you do not have that information; do not answer from general
      knowledge and do not guess.",
    "created_at": "2026-11-06",
    "use_count": 0,
    "failure_type": "UNSUPPORTED_ANSWER",
    "applies_when": [
      "no_data"
    ],
    "source": "training failures: tra_07, tra_12"
  },
  {
    "lesson_id": "lesson_002",
    "failure_pattern": "(controlled test) Multi-part questions produce long answers.",
    "correction": "If the question asks about more than one thing, answer only the first item and say the rest cannot be confirmed.",
    "created_at": "2026-11-06",
    "use_count": 0,
    "failure_type": "CONTROLLED_OVERBROAD",
    "applies_when": [
      "multi_part"
    ],
    "source": "controlled: deliberately harmful, to demonstrate the validation gate"
  }
]
```


Reuse demonstration, later question "Who painted the Mona Lisa?": without lessons the answer was "The Mona Lisa was painted by Leonardo da Vinci." (REVISE, UNSUPPORTED_ANSWER); with the store loaded (lessons used: lesson_001) it was "I do not have that information." (PASS). use_count went {"lesson_001": 0} -> {"lesson_001": 1} and the new value is in the file ({"lesson_001": 1}).

4.3 Validation before keeping a lesson

Keep a lesson only if: no previously correct answer becomes wrong; at least one failing case is fixed; total failures do not increase; unsupported answers do not increase.

The rejected lesson ("controlled: deliberately harmful, to demonstrate the validation gate") told the agent to answer only the first part of a multi-part question. On val_02 it changed a previously correct answer into a WRONG_FACT ('The fixture FX-9871-00910 is between Oakland Lions and Santa Clara Bears, and there are 22 spots left.'), so it was removed, as the assignment requires. Candidates: 2; kept: 1. test_data.json was not used to create or choose any lesson.

Candidate lesson	Source	Fixed	Regressions	Failures before -> after	Unsupported before -> after	Decision
lesson_001 (UNSUPPORTED_ANSWER)	training failures: tra_07, tra_12	val_04, val_12	none	2 -> 0	2 -> 0	**KEEP**
lesson_002 (CONTROLLED_OVERBROAD)	controlled: deliberately harmful, to demonstrate the validation gate	none	val_02	0 -> 1	0 -> 0	**REJECT**

```
   checks = {"no previously correct answer became wrong": not regress, "at least one failing case fixed":
bool(fixed),
             "failures did not increase": fail_a <= fail_b, "unsupported answers did not increase": unsup_a
<= unsup_b}
   keep = all(checks.values())
```

4.4 Four variants on the final test set

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	15 / 18	3	1	998.5
Evaluator Repair	18 / 18	0	1.17	1011.7
Lesson Memory	18 / 18	0	1.11	1049.2
Validated Improvement	18 / 18	0	1	839.4

A question counts as correct when the Evaluator's final verdict is PASS. First-answer failures per variant:

Variant	Questions whose FIRST answer failed	Failure type
Baseline	tes_04, tes_05, tes_15	UNSUPPORTED_ANSWER
Evaluator Repair	tes_04, tes_05, tes_15	UNSUPPORTED_ANSWER
Lesson Memory	tes_06, tes_08	OMITTED_FACT
Validated Improvement	none	-

What this shows: without feedback the baseline gets 15 of 18 right; its 3 errors are all the same kind: off-topic questions ("Translate 'good morning' into Spanish."; "Explain how photosynthesis works."; "What is the tallest mountain in the world?") answered from general knowledge instead of saying the data is not available. The Evaluator's feedback repaired all of them on the second attempt (Evaluator Repair 18/18), at the cost of 1.17 attempts per question on average. The validated lesson lets the first answer be right (Validated Improvement 18/18, average attempts 1). Lesson Memory, which loads every proposed lesson without validation, also ends with 18/18 correct but needed 1.11 attempts, because it also loaded the harmful controlled lesson (lesson_002, which was loaded on tes_06, tes_08; those needed a second attempt: the Evaluator caught the omitted part and the repair loop fixed it, so the damage shows up as extra attempts and latency, not as a wrong final answer). Average latency is nearly the same for Baseline and Evaluator Repair (998.5 and 1011.7 ms); the lower figure for Validated Improvement (839.4 ms) is mostly because the refusals it now gives are very short answers, not a speed-up of the model. With 18 questions, a one-question difference is not strong evidence; qwen3 is not bit-reproducible, so another run can differ slightly.

Console output of run_part4.py (my own PowerShell)

```
PS C:\Users\patel\data260-9871>   cd C:\Users\patel\data260-9871\code\agents_hw9
PS C:\Users\patel\data260-9871\code\agents_hw9>   ..\venv\Scripts\python.exe run_part4.py
```

```
== training run (baseline, no lessons) ==
training: 12/14 correct, failures=[('tra_07', 'UNSUPPORTED_ANSWER'), ('tra_12', 'UNSUPPORTED_ANSWER')]
candidate lessons: [('lesson_001', 'UNSUPPORTED_ANSWER'), ('lesson_002', 'CONTROLLED_OVERBROAD')]
== validation (single attempt, so a lesson's effect is not hidden by the repair loop): system without any lesson ==
== validation: add lesson_001 (UNSUPPORTED_ANSWER) ==
   KEEP lesson_001: fixed=['val_04', 'val_12'] regressions=[] failures 2->0 unsupported 2->0
== validation: add lesson_002 (CONTROLLED_OVERBROAD) ==
   REJECT lesson_002: fixed=[] regressions=['val_02'] failures 0->1 unsupported 0->0
kept lessons: ['lesson_001']
== final test: baseline (18 questions) ==
   {'questions': 18, 'correct': 15, 'incorrect': 3, 'avg_attempts': 1, 'avg_latency_ms': 998.5}
== final test: evaluator_repair (18 questions) ==
   {'questions': 18, 'correct': 18, 'incorrect': 0, 'avg_attempts': 1.17, 'avg_latency_ms': 1011.7}
== final test: lesson_memory (18 questions) ==
   {'questions': 18, 'correct': 18, 'incorrect': 0, 'avg_attempts': 1.11, 'avg_latency_ms': 1049.2}
== final test: validated (18 questions) ==
   {'questions': 18, 'correct': 18, 'incorrect': 0, 'avg_attempts': 1, 'avg_latency_ms': 839.4}
== demo 1: revision sequence ==
```

Short explanations

- Agent communication: the controller passes plain JSON messages. To the Answering Agent: question, attempt number, and (on attempt 2) the Evaluator's feedback and its previous answer. From it: the answer and the lessons it used. To the Evaluator Agent: question and answer. From it: PASS or REVISE, an explanation, feedback and a failure type. The agents never share state other than these messages.
- Lesson reuse: before each answer the Answering Agent computes tags for the question (multi-part, no recorded data, names a fixture), loads the lessons whose `applies_when` matches, adds their corrections to its prompt and increments `use_count`.
- Stopping condition: at most two attempts. A PASS stops immediately; a REVISE after attempt 1 triggers one revision; after attempt 2 the system stops and reports the result whatever it is.
- Validation process: each candidate is added to the kept set, the validation questions are run before and after, and the four rules above decide. The test set is run only afterwards, once per variant.
- Limitation: the Evaluator recognises what a question asks with keyword patterns and checks the answer by string matching, so an unusual phrasing or a differently worded correct answer can be mis-scored; only 18 test questions were used, and the failures are concentrated in one pattern (answering off-topic questions), so the results do not show how the lessons would behave on other kinds of failure.

Self-check (verification.json)

`verify_hw09.py` runs the real system and 13 objective checks (commit `6c2ef77`, SEED 9871, VERIFY_SEED 269871): 13/13 passed. It was also run against three deliberately broken states and failed where it should (`raw/verify_broken_states.txt`: a missing file -> 9/12, Redis stopped -> 7/12, test_data.json edited after commit -> 11/12).

Check	Result
required files exist	PASS
Services - Redis Stack (RediSearch) and MySQL s9871_rel are up	PASS
P1 - Redis counts every access, MySQL syncs only at the 10th (read directly), s9871 key prefix	PASS
P2 - semantic cache - miss, duplicate hit, paraphrase hit, unrelated miss, rejected answer not cached, hit/miss logged	PASS
P3 - the 40 labels in cache_cases.yaml agree with the database	PASS
P3 - the threshold sweep reproduces the committed hit and false-hit counts	PASS
P3 - stale cached answer served without the version check, invalidated with it	PASS
P4 - offline agent/evaluator/controller/lesson tests	PASS
P4 - dataset files unchanged since commit; inputs hold only id + question	PASS
P4 - lesson validation decisions follow the four rules; lessons.json holds only the kept lessons	PASS
P4 - results table matches the saved per-question results	PASS
P4 - REAL model - forced incomplete answer is repaired in 2 attempts; off-topic question declined with the kept lesson	PASS
Ruff passes on the HW9 code	PASS

AI_USE

1. What did you use an AI assistant for, and what did you do yourself?

I used Claude Code mainly as a tutor and debugging assistant. I asked it to explain concepts related to Redis, semantic caching, stale-answer invalidation, threshold selection, cost modeling, and the Answering and Evaluator agents. I also used it when I was unsure how to begin, encountered an error, or needed help understanding why something was not working. I wrote and implemented the code myself. I ran the tests and measurement scripts on my own machine, reviewed the output and results, took the required screenshots, decided which results to include in the report, and made the Git commits myself.

2. One mistake, how it was caught, and how it was fixed

The first version of my lesson-validation step used the Evaluator's repair loop while testing the validation questions. Since the repair loop corrected every wrong first answer, all of the validation questions became correct both before and after loading the lesson. This meant that the validation step could not measure whether the lesson actually improved the model's first answer.

3. How did you detect the problem or verify the result?

I ran the complete pipeline with the real model before the reported run. The dry run showed zero failures before and after applying both candidate lessons, even though the training run had failed twice. This showed that the validation results were not meaningful because the repair loop was hiding the lesson's actual effect.

4. What did you change, and why does it work now?

I changed the validation process so that each question is answered only once without the repair loop, while still loading the candidate lesson. This measures the lesson's effect on the model's initial answer. After the change, the real lesson was kept because it fixed `val_04` and `val_12` without causing regressions. The deliberately harmful lesson was rejected because it caused `val_02` to fail.